

Spooling

Cómo convertir un dispositivo lento y exclusivo
en un recurso concurrente y eficiente

SPOOL = *Simultaneous Peripheral Operation On-Line*

El problema: dispositivos exclusivos

Definición

Un **dispositivo exclusivo** (también *dedicado* o *serial*) solo puede ser utilizado por **un proceso a la vez**. La impresora es el caso clásico: mientras un proceso imprime, ningún otro puede usarla.

Sin spooling, si el proceso A manda a imprimir un documento de 5 minutos, los procesos B y C que también quieren imprimir quedan **bloqueados**, esperando su turno. Esto genera tres problemas:

- **Ineficiencia:** el dispositivo queda inactivo cada vez que espera datos del CPU, y el CPU queda inactivo cada vez que espera al dispositivo.
- **Bloqueo:** los demás procesos deben esperar, sin poder avanzar en otras tareas que nada tienen que ver con imprimir.
- **Pérdida de concurrencia:** el sistema deja de comportarse como multitarea; todo se serializa alrededor del dispositivo lento.

Proceso A

imprime documento (5 min)

Proceso B

bloqueado, espera forzada

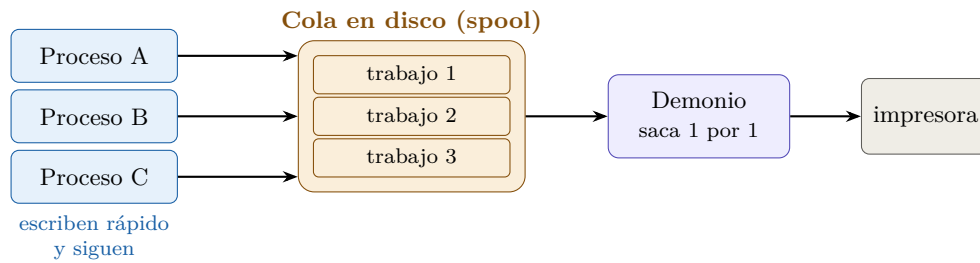
Proceso C

bloqueado, espera forzada

La solución: el spooler

Idea clave

El spooler mete un **buffer (cola) en disco** entre los procesos y el dispositivo. En vez de hablar con la impresora directamente y bloquearse, cada proceso escribe su trabajo *rápido* en la cola de disco y **continúa**. Un único gestor en segundo plano, el **demonio** (*daemon*) de spooling, saca los trabajos de la cola uno por uno y alimenta la impresora a su ritmo lento.



El cambio esencial: los procesos ya **no hablan con la impresora**, hablan con la **cola en disco**, que es rapidísima. Como escribir en disco toma milisegundos y la impresora toma minutos, cada proceso entrega su trabajo y sigue de inmediato. El demonio es quien se encarga de la parte lenta, en segundo plano.

Antes y después

SIN spooling	CON spooling
A imprime (5 min); B y C bloqueados esperando	A, B y C encolan su trabajo y siguen trabajando
✗ el dispositivo queda ocioso esperando al CPU	✓ sin esperas bloqueantes
✗ el CPU gasta ciclos esperando la E/S	✓ el CPU queda libre para otras tareas
✗ B y C paralizados indefinidamente	✓ gestión central de la cola
✗ se pierde la concurrencia	✓ parece concurrente y eficiente

Los cuatro beneficios

1. **Múltiples procesos sin esperas bloqueantes.** Cada proceso escribe en la cola de disco (rapidísimo) y sigue. La “fila” ahora es de *trabajos en disco*, no de *procesos bloqueados*.
2. **El CPU no gasta ciclos esperando E/S lenta.** El proceso solo hace la escritura rápida al disco y el CPU queda libre para otros procesos. El único que espera a la impresora es el demonio, que lo hace de forma eficiente (dormido hasta que hay trabajo).
3. **Gestión centralizada con políticas inteligentes.** Como todos los trabajos pasan por una sola cola gestionada por el demonio, ahí se aplican políticas: prioridades (un trabajo urgente primero), límites (que nadie acapare la impresora), agrupación, etc. Un solo punto de control.
4. **Mayor fiabilidad ante fallos parciales.** Los trabajos están **guardados en disco**, no solo en la memoria volátil del proceso. Si la impresora se atasca o el sistema se reinicia, los trabajos **no se pierden**: siguen en la cola y se reanudan cuando el dispositivo vuelve.

En resumen

El spooling es un caso del patrón **productor–consumidor con buffer**: los procesos son productores rápidos, la impresora es un consumidor lento, y el disco es el buffer que los *desacopla*. Ese truco — meter un buffer intermedio para que un lado rápido no espere a un lado lento — aparece en muchísimos lugares de los sistemas operativos. El nombre SPOOL (*Simultaneous Peripheral Operation On-Line*) viene de que la operación del periférico ocurre “en línea” y de forma **simultánea** al trabajo de los procesos, en vez de bloquearlos.